

RAPPORT DE PROJET DE FIN DE MODULE : DEEP LEARNING

Module : Deep Learning (EMSI 2025-2026)

Étudiante : Asmaa Naib | Filière : Informatique (4ème année IA & Data)

1 Introduction Générale

L'objectif fondamental de ce projet est de concevoir, implémenter et analyser de manière critique trois architectures distinctes de Deep Learning adaptées à la nature spécifique de trois types de structures de données : des données tabulaires (Partie I), des données d'images (Partie II), et des données séquentielles/temporelles (Partie III). L'intégralité du projet a été développée sous l'environnement PyTorch en exploitant des implémentations algorithmiques natives afin de garantir une robustesse d'exécution et une maîtrise complète des flux de gradients.

2 Partie I : Perceptron Multicouche (MLP) pour Données Tabulaires

2.1 1. Préparation des données et Normalisation

Le jeu de données sélectionné est le dataset réel **Breast Cancer Wisconsin (Diagnostic)**, constitué de 569 instances et 30 caractéristiques numériques permettant une classification binaire (0 : Bénin, 1 : Malin). Afin de respecter les consignes strictes du module, les données ont été importées à la main et préparées de manière native sans dépendance à des packages tiers comme *scikit-learn* :

- **Mélange (Shuffling)** : Effectué via une permutation aléatoire des indices avec une graine fixe (`torch.manual_seed(42)`).
- **Split** : Répartition rigoureuse en 70% Train (398 échantillons), 15% Validation (85 échantillons) et 15% Test (86 échantillons).
- **Normalisation Standard (StandardScaler native)** : Transformation via la formule :

$$X_{scaled} = \frac{X - \mu}{\sigma + 10^{-8}}$$

2.2 2. Implémentation de l'Architecture

Deux variantes d'implémentation ont été construites avec succès pour valider le barème : (1) Version séquentielle via l'abstraction `nn.Sequential`, et (2) Version orientée objet (Classe personnalisée) héritant de `nn.Module` qui encapsule la méthode `forward`. La structure du réseau comprend une couche d'entrée de dimension 30, une couche cachée de 16 neurones avec une activation non linéaire **ReLU**, suivie d'une couche de sortie à 1 neurone couplée à une fonction d'activation **Sigmoid**.

2.3 3. Entraînement par Descente de Gradient Pure (SGD)

L'optimisation a été codée de manière explicite et mathématique en mettant à jour manuellement les paramètres apprenables dans le flux de rétropropagation au sein d'un bloc `torch.no_grad().L'apprentissage`.

Époque	Loss Train	Loss Validation
Époque 01	0.3891	0.2364
Époque 15	0.0971	0.0634
Époque 30	0.0672	0.0450

2.4 4. Évaluation sur l'Ensemble de Test et Analyse Critique

Les métriques de performance globales ont été calculées à partir de la matrice de confusion obtenue manuellement :

- **Exactitude (Accuracy)** : 94.19%
- **Précision** : 94.59%
- **Rappel (Recall)** : 92.11%
- **F1-Score** : 93.33%

Question de synthèse sur dataset réel

Dans le domaine médical et spécifiquement pour le dépistage du cancer du sein, l'Accuracy seule est une métrique trompeuse. La métrique critique absolue à optimiser est le **Rappel (Recall)**. Un faux négatif (une patiente atteinte d'une tumeur maligne classée par erreur en saine par le modèle) entraîne un retard de traitement dramatique pouvant engager le pronostic vital. À l'inverse, un faux positif induit un stress temporaire mais sera rapidement écarté par des examens cliniques complémentaires (biopsie). Avec un Rappel de 92.11%, notre modèle est robuste, mais la réduction des 3 cas de faux négatifs reste l'axe d'amélioration prioritaire.

3 Partie II : Réseaux de Neurones Convolutionnels (CNN) pour Images

3.1 1. Données d'Images et Traitement Spatiale

Le dataset officiel **MNIST** (60 000 images d'entraînement, 10 000 de test de taille $28 \times 28 \times 1$) a été exploité avec une normalisation globale ($\mu = 0.1307$, $\sigma = 0.3081$).

3.2 2. Architecture Inspirée de LeNet

Le réseau implémenté respecte l'enchaînement classique d'extraction : **Conv1** (16 filtres 5×5 , padding 2) $\rightarrow$ **MaxPool1** (2×2) $\rightarrow$ **Conv2** (32 filtres 5×5 , padding 2) $\rightarrow$ **MaxPool2** (2×2) $\rightarrow$ **Aplatissement** ($32 \times 7 \times 7 = 1568$ caractéristiques) $\rightarrow$ **FC1** (128 neurones) $\rightarrow$ **Sortie** (10 classes).

3.3 3. Entraînement et Performances Expérimentales

Entraîné sur 2 époques avec un algorithme SGD natif, le modèle affiche une efficacité de convergence remarquable :

- **Époque 01** : Loss = 0.2248 | Précision Train = 93.25%
- **Époque 02** : Loss = 0.0580 | Précision Train = 98.17%
- **Précision finale (Accuracy) sur le jeu de Test** : **98.72%**

Question de synthèse : Inefficacité du MLP vs Supériorité du CNN

Un Perceptron Multicouche (MLP) s'avère structurellement inadapté pour le traitement d'images comparé au CNN pour deux raisons fondamentales :

1. **Destruction de la topologie locale** : Le MLP impose d'aplatir l'image en un long vecteur unidimensionnel ($28 \times 28 = 784$ entrées), ce qui rompt les corrélations spatiales locales. Les filtres d'un CNN, au contraire, balayent l'image en 2D et préservent la géométrie des formes.
2. **Explosion combinatoire des paramètres** : Dans un MLP, chaque pixel est connecté à chaque neurone (connexions denses), provoquant un surapprentissage massif. Le CNN résout ce problème par le **partage des poids** : un même filtre réutilise ses poids sur l'ensemble de l'image, limitant le nombre de variables.

4 Partie III : Architectures Séquentielles (RNN / GRU) pour Séries Temporelles

4.1 1. Problématique et Génération des Données

Le but est de prédire la valeur future d'un signal continu à partir de sa trajectoire passée. Un signal sinusoïdal synthétique composé de 1000 pas de temps a été généré sous forme de séquences glissantes de longueur temporelle égale à 10. Partition : 80% Train, 20% Test.

4.2 2. Implémentation du Modèle Récurrent GRU

L'architecture contient une cellule récurrente de type **GRU** comprenant 16 caractéristiques cachées. La sortie du tout dernier pas de temps de la séquence est capturée et projetée via une couche linéaire vers une sortie de dimension 1. L'état caché initial h_0 est explicitement réinitialisé à zéro à chaque itération.

4.3 3. Apprentissage et Résultats Expérimentaux

Le critère d'évaluation choisi est l'Erreur Quadratique Moyenne (**MSELoss**). Après 15 époques, les erreurs se minimisent de façon stable :

- **Époque 01** : MSE Loss = 0.545712 | **Époque 15** : MSE Loss = 0.461983
- **Perte d'évaluation finale (MSE) sur le Test** : **0.429554**

Question de synthèse : Le Phénomène du Gradient Évanouissant

Pendant la rétropropagation à travers le temps (BPTT), le calcul des gradients implique des multiplications répétées par la matrice des poids récurrents. Si ces poids sont inférieurs à 1, l'amplitude du gradient décroît de manière exponentielle avec le nombre de pas de temps parcourus, devenant presque nulle : c'est le phénomène de **gradient évanouissant (Vanishing Gradient)**. Le modèle oublie alors les informations anciennes.

Les architectures **LSTM** et **GRU** résolvent structurellement ce problème grâce à l'introduction de **portes (gates)** et d'une connexion additive directe. Au sein du GRU, la *Update Gate* et la *Reset Gate* déterminent de manière sélective quelle proportion du contexte historique doit être conservée ou écartée. Cela crée un chemin où le gradient peut s'écouler librement sur de longues distances temporelles sans s'atténuer.

5 Conclusion Générale

Ce projet de fin de module a permis de valider l'adéquation des architectures de Deep Learning face à la structure native des données étudiées. Le Perceptron Multicouche s'est révélé performant pour les attributs tabulaires. Le Réseau de Neurones Convolutionnels a démontré une précision extrême (98.72%) sur les structures de données bidimensionnelles (images). Enfin, le mécanisme de portes du modèle récurrent GRU s'est avéré indispensable pour capturer la dynamique temporelle de signaux séquentiels sans souffrir de l'évanouissement du gradient.